

SCRIPTS SQL – HUMAN RESOURCES

Muestra todos los registros y columnas de EMPLOYEES

```
SELECT *  
FROM EMPLOYEES;
```

Muestra nombre, apellido, salario y el 10% del salario

```
SELECT FIRST_NAME,  
       LAST_NAME,  
       SALARY,  
       SALARY*0.1  
FROM EMPLOYEES;
```

Calcula el salario por comisión

```
SELECT FIRST_NAME,  
       LAST_NAME,  
       SALARY,  
       SALARY*0.1,  
       SALARY*COMMISSION_PCT  
FROM EMPLOYEES;
```

Uso de alias simples para columnas

```
SELECT FIRST_NAME AS nombre,  
       LAST_NAME AS apellido,  
       SALARY,  
       SALARY*0.1 AS por_de_salario,  
       SALARY*COMMISSION_PCT AS sal_por_comm  
FROM EMPLOYEES;
```

Alias usando comillas para nombres con espacios

```
SELECT FIRST_NAME AS "nombre",  
       LAST_NAME AS "apellido",  
       SALARY AS "salario",  
       SALARY*0.1 AS "porcentaje de salario",  
       SALARY*COMMISSION_PCT AS "sal_por_comm"  
FROM EMPLOYEES;
```

Concatenación de apellido y JOB_ID

```
SELECT FIRST_NAME AS "nombre",  
       LAST_NAME AS "apellido",  
       SALARY AS "salario",  
       SALARY*0.1 AS "porcentaje de salario",  
       SALARY*COMMISSION_PCT AS "sal_por_comm",  
       LAST_NAME||JOB_ID  
FROM EMPLOYEES;
```

Une apellido y JOB_ID con un espacio

```
SELECT LAST_NAME|| ' ' ||JOB_ID  
FROM EMPLOYEES;
```

Uso de Q'[]' para manejar comillas simples en texto

```
SELECT LAST_NAME|| Q'[ IS' A ]' ||JOB_ID AS TABLA  
FROM EMPLOYEES;
```

Muestra REGION_ID sin repetir valores (DISTINCT)

```
SELECT DISTINCT REGION_ID  
FROM COUNTRIES;
```

Busca empleados con apellido King (FILTROS CON WHERE)

```
SELECT *  
FROM EMPLOYEES  
WHERE LAST_NAME = 'King';
```

Empleados contratados después de 2010

```
SELECT *  
FROM EMPLOYEES  
WHERE hire_date > '01-01-2010';
```

Empleados contratados después de 2014

```
SELECT *  
FROM EMPLOYEES  
WHERE hire_date > '01-01-2014';
```

Nombres que empiezan con A (LIKE)

```
SELECT *  
FROM EMPLOYEES  
WHERE first_name LIKE 'A%';
```

Nombres que empiezan con An

```
SELECT *  
FROM EMPLOYEES  
WHERE first_name LIKE 'An%';
```

Nombres que terminan en n

```
SELECT *  
FROM EMPLOYEES  
WHERE first_name LIKE '%n';
```

Nombres que contienen n

```
SELECT *  
FROM EMPLOYEES  
WHERE first_name LIKE '%n%';
```

Segunda letra = o

```
SELECT *  
FROM EMPLOYEES  
WHERE first_name LIKE '_o%';
```

Empleados que tienen comisión (NULL)

```
SELECT *  
FROM EMPLOYEES  
WHERE COMMISSION_PCT IS NOT NULL;
```

Empleados sin comisión

```
SELECT *  
FROM EMPLOYEES  
WHERE COMMISSION_PCT IS NULL;
```

Salario mayor a 5000 y departamento 90

```
SELECT *  
FROM EMPLOYEES  
WHERE SALARY > 5000  
AND DEPARTMENT_ID = 90;
```

Salario mayor a 5000 o departamento 80

```
SELECT *  
FROM EMPLOYEES  
WHERE SALARY > 5000  
OR DEPARTMENT_ID = 80;
```

Ordena empleados por salario descendente (ORDER BY)

```
SELECT *  
FROM EMPLOYEES  
ORDER BY SALARY DESC;
```

Salarios entre 12000 y 15000 (BETWEEN)

```
SELECT *  
FROM EMPLOYEES  
WHERE SALARY BETWEEN 12000 AND 15000;
```

Contratados entre 2012 y 2021

```
SELECT *  
FROM EMPLOYEES  
WHERE HIRE_DATE BETWEEN '01-01-2012' AND '31-12-2021';
```

Contratados fuera del rango 2012-2021

```
SELECT *  
FROM EMPLOYEES  
WHERE HIRE_DATE NOT BETWEEN '01-01-2012' AND '31-12-2021';
```

Departamentos específicos (IN)

```
SELECT *  
FROM EMPLOYEES  
WHERE DEPARTMENT_ID IN (90, 80, 100, 60);
```

Países de regiones 1 y 2

```
SELECT *  
FROM COUNTRIES  
WHERE REGION_ID IN (1, 2);
```

Países de regiones 10 y 20

```
SELECT *  
FROM COUNTRIES  
WHERE REGION_ID IN (10, 20);
```

Empleados con manager

```
SELECT *  
FROM EMPLOYEES  
WHERE MANAGER_ID IS NOT NULL;
```

Empleados sin manager

```
SELECT *  
FROM EMPLOYEES  
WHERE MANAGER_ID IS NULL;
```

Busca apellido exacto Reina

```
SELECT *  
FROM EMPLOYEES  
WHERE LAST_NAME='Reina';
```

Diferencias mayúsculas/minúsculas

```
SELECT *  
FROM EMPLOYEES  
WHERE LAST_NAME='REINA';
```

Solicita valor una sola vez

```
SELECT *  
FROM EMPLOYEES  
WHERE LAST_NAME='&KING';
```

Solicita valor y lo reutiliza

```
SELECT *  
FROM EMPLOYEES  
WHERE LAST_NAME='&&KING';
```

Permite escribir cualquier condición.EJEMPLO: 1=1 devuelve todos los registros

```
SELECT *  
FROM EMPLOYEES  
WHERE &CONDICION;
```

Convierte nombres a mayúsculas y apellidos a minúsculas

```
SELECT FIRST_NAME,  
       UPPER(FIRST_NAME),  
       LAST_NAME,  
       LOWER(LAST_NAME)  
FROM EMPLOYEES;
```

Concatena nombre y apellido formateados

```
SELECT UPPER(FIRST_NAME)|| ' ' || LOWER(LAST_NAME)  
FROM EMPLOYEES;
```

Nombre completo con alias

```
SELECT UPPER(FIRST_NAME) || ' ' || LOWER(LAST_NAME) AS  
NOMBRE_COMPLETO  
FROM EMPLOYEES;
```

ROUND con diferentes precisiones

```
SELECT ROUND(45.923,2),  
       ROUND(45.923,0),  
       ROUND(45.923,-1)  
FROM DUAL;
```

Empleados contratados antes de febrero 2013 (FECHAS)

```
SELECT last_name,  
       hire_date  
FROM employees  
WHERE hire_date < '01-FEB-2013';
```

Diferencia entre formatos YY y RR

```
SELECT  
'22/09/2006' AS "FECHA_NORMAL",  
TO_DATE('22/09/2006','DD/MM/YY') AS "FECHA_YY",  
TO_DATE('22/09/2006','DD/MM/RR') AS "FECHA_RR"  
FROM DUAL;
```

Zona horaria y fecha actual

```
SELECT SESSIONTIMEZONE,  
       CURRENT_DATE  
FROM DUAL;
```

Zona horaria y timestamp actual

```
SELECT SESSIONTIMEZONE,  
       CURRENT_TIMESTAMP  
FROM DUAL;
```

Redondea 555 a miles

```
SELECT ROUND(555, -3)  
FROM DUAL;
```

Redondea 15 a decenas

```
SELECT ROUND(15, -1)  
FROM DUAL;
```

Elimina cifras a nivel decenas

```
SELECT TRUNC(15.9457, -1)  
FROM DUAL;
```

Suma un día a hire_date

```
SELECT last_name,  
       hire_date,  
       hire_date + 1 AS fecha_ajustada  
FROM employees;
```

Conversión correcta usando YYYY

```
SELECT '15/05/2007',  
       TO_DATE('15/05/2007', 'DD/MM/YYYY') AS YY  
FROM DUAL;
```

Uso de RR para años de 2 dígitos

```
SELECT '15/05/07',  
       TO_DATE('15/05/07', 'DD/MM/RR') AS RR  
FROM DUAL;
```

Fecha y hora del servidor

```
SELECT SYSDATE  
FROM DUAL;
```

Fecha actual de sesión

```
SELECT SESSIONTIMEZONE,  
       CURRENT_DATE  
FROM DUAL;
```

Diferencia en días desde 15/05/2007

```
SELECT SESSIONTIMEZONE,  
       CURRENT_DATE - TO_DATE('15/05/2007', 'DD/MM/YYYY') AS  
diferencia_dias  
FROM DUAL;
```

Diferencia convertida a segundos

```
SELECT ROUND(  
    (CURRENT_DATE - TO_DATE('15/05/2007', 'DD/MM/YYYY')) * 86400,0) AS  
segundos  
FROM DUAL;
```

Redondea fecha al inicio de mes

```
SELECT ROUND(  
    TO_DATE('25/04/26', 'DD/MM/YY') - 2,  
    'MM'  
    ) - 1 AS fecha  
FROM DUAL;
```

Calcula meses transcurridos hasta hoy

```
SELECT ROUND(  
    MONTHS_BETWEEN(  
        CURRENT_DATE,  
        TO_DATE('15/05/07', 'DD/MM/RR')),0) AS meses_vividos  
FROM DUAL;
```

Agrega un mes a la fecha actual

```
SELECT ADD_MONTHS(CURRENT_DATE, 1)  
FROM DUAL;
```

Último día del mes actual

```
SELECT LAST_DAY(CURRENT_DATE)  
FROM DUAL;
```

Último día dentro de 5 meses

```
SELECT LAST_DAY(  
    ADD_MONTHS(CURRENT_DATE, 5)  
    )  
FROM DUAL;
```

Muestra la fecha del próximo viernes después del 12/05/2026

```
SELECT NEXT_DAY(DATE '2026-05-12', 'FRIDAY')  
FROM DUAL;
```

Muestra último viernes del mes según la cantidad de meses indicada en &MES

```
SELECT NEXT_DAY(LAST_DAY(ADD_MONTHS(CURRENT_DATE, &MES))-  
7,'VIERNES')  
FROM DUAL;
```

-- INNER JOIN:

-- Une las dos tablas tomando SOLO los registros que coinciden

-- en ambas tablas según la condición del ON.

-- En este caso, trae empleados que tienen un departamento válido.

```
SELECT E.FIRST_NAME, D.DEPARTMENT_NAME  
FROM EMPLOYEES E  
INNER JOIN DEPARTMENTS D  
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

-- INNER JOIN:

-- Busca coincidencias entre EMPLOYEES y DEPARTMENTS.

-- Aquí la condición está fija en E.DEPARTMENT_ID = 1,

-- por lo tanto solo se relacionan empleados del departamento 1.

```
SELECT E.FIRST_NAME, D.DEPARTMENT_NAME  
FROM EMPLOYEES E  
INNER JOIN DEPARTMENTS D  
ON E.DEPARTMENT_ID = 1;
```

-- LEFT JOIN:

-- Toma TODOS los registros de la tabla izquierda (EMPLOYEES)

-- y solo los coincidentes de la tabla derecha (DEPARTMENTS).

-- Si un empleado no tiene departamento, el departamento sale NULL.

```
SELECT E.FIRST_NAME, D.DEPARTMENT_NAME  
FROM EMPLOYEES E  
LEFT JOIN DEPARTMENTS D  
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

-- RIGHT JOIN:

-- Toma TODOS los registros de la tabla derecha (DEPARTMENTS)

-- y solo los coincidentes de la izquierda (EMPLOYEES).

-- Si un departamento no tiene empleados, los datos del empleado salen NULL.

```
SELECT E.FIRST_NAME, D.DEPARTMENT_NAME  
FROM EMPLOYEES E  
RIGHT JOIN DEPARTMENTS D  
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

-- NATURAL JOIN:

-- Une automáticamente las tablas usando columnas

-- que tengan el mismo nombre en ambas tablas.

-- En este caso normalmente usa DEPARTMENT_ID.

```
SELECT *  
FROM EMPLOYEES NATURAL JOIN DEPARTMENTS;
```

-- FULL OUTER JOIN:

-- Trae TODOS los registros de ambas tablas.

-- Muestra coincidencias y también registros sin relación.

-- Si no hay coincidencia, las columnas faltantes salen NULL.

```
SELECT E.FIRST_NAME, D.DEPARTMENT_NAME
```



```
FROM EMPLOYEES E
FULL OUTER JOIN DEPARTMENTS D
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID;
```

-- CROSS JOIN:

-- Hace un producto cartesiano.

-- Cada empleado se combina con TODOS los departamentos.

-- No necesita condición ON.

```
SELECT *
FROM EMPLOYEES CROSS JOIN DEPARTMENTS;
```

-- INNER JOIN múltiple:

-- Va uniendo tablas relacionadas paso a paso.

-- Solo muestra registros que tengan coincidencia en TODAS las tablas.

-- Sirve para obtener la región donde trabaja cada empleado.

```
SELECT
    E.FIRST_NAME || ' ' || E.LAST_NAME AS EMPLEADO,
    R.REGION_NAME AS REGION
FROM EMPLOYEES E
INNER JOIN DEPARTMENTS D
    ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
INNER JOIN LOCATIONS L
    ON D.LOCATION_ID = L.LOCATION_ID
INNER JOIN COUNTRIES C
    ON L.COUNTRY_ID = C.COUNTRY_ID
INNER JOIN REGIONS R
    ON C.REGION_ID = R.REGION_ID;
```

-- SELF JOIN:

-- La tabla EMPLOYEES se une consigo misma.

-- Una copia representa empleados y la otra manager.

-- Sirve para relacionar empleados con sus jefes.

```
SELECT E.FIRST_NAME AS EMPLEADO,
    M.FIRST_NAME AS MANAGER
FROM EMPLOYEES E
INNER JOIN EMPLOYEES M
ON E.MANAGER_ID = M.EMPLOYEE_ID;
```

-- SELF JOIN + WHERE:

-- Igual que el anterior, pero filtrando

-- empleados cuyo manager se llama Steven.

```
SELECT E.FIRST_NAME AS EMPLEADO,
    M.FIRST_NAME AS MANAGER
FROM EMPLOYEES E
INNER JOIN EMPLOYEES M
ON E.MANAGER_ID = M.EMPLOYEE_ID
```

```
WHERE M.FIRST_NAME = 'Steven';
```

-- SELF JOIN ordenado:

-- Relaciona empleados con managers

-- y ordena el resultado por nombre del manager.

```
SELECT E.FIRST_NAME AS EMPLEADO,  
       M.FIRST_NAME AS MANAGER  
FROM EMPLOYEES E  
INNER JOIN EMPLOYEES M  
  ON E.MANAGER_ID = M.EMPLOYEE_ID  
ORDER BY M.FIRST_NAME ASC;
```

COUNT(*): Cuenta todos los registros de la tabla EMPLOYEES.

```
SELECT COUNT(*)  
FROM EMPLOYEES;
```

Cuenta todas las filas porque 'X' nunca es NULL.

```
SELECT COUNT('X') AS CUENTA  
FROM EMPLOYEES;
```

MAX: Obtiene el salario más alto de la tabla.

```
SELECT MAX(SALARY) AS SALARIO_MAXIMO  
FROM EMPLOYEES;
```

-- GROUP BY:

Agrupa empleados por departamento y obtiene el salario máximo de cada grupo.

```
SELECT DEPARTMENT_ID, MAX(SALARY)  
FROM EMPLOYEES  
GROUP BY DEPARTMENT_ID  
ORDER BY MAX(SALARY) DESC;
```

Agrupa empleados por departamento y calcula:

```
SELECT DEPARTMENT_ID,  
       MAX(SALARY),  
       MIN(SALARY),  
       ROUND(AVG(SALARY),2) AS PROMEDIO_SALARIO,  
       COUNT(EMPLOYEE_ID)  
FROM EMPLOYEES  
GROUP BY DEPARTMENT_ID  
ORDER BY MAX(SALARY) DESC;
```